Цепи Маркова

Цепи Маркова

Идея: вероятностные переходы

В конечных автоматах переходы **детерминированные**: если враг видит игрока - переход в Chase. Всегда. Но что, если мы хотим, чтобы враг *иногда* преследовал, *иногда* убегал, а *иногда* продолжал патрулировать? Не по жёстким правилам, а по **вероятностям**?

Ну, первая мысль - раскидать `random()` по условиям автомата:

```
if see_player:
    if random() < 0.3: state = CHASE
    elif random() < 0.5: state = FLEE
    # else stay in PATROL... probably?
```

Хм. Каждый `random()` - отдельный бросок, вероятности не складываются в 1.0, а перебалансировать 4 состояния в таких `if-elif` - мучение. А что, если собрать все вероятности в одну таблицу?

Цепь Маркова - это конечный автомат, в котором переходы задаются не условиями, а **вероятностями**. Из состояния "Patrol" враг с вероятностью 50% остаётся на месте, с 30% переходит в Chase, с 20% - в Flee. Каждый тик - бросок кости.

Матрица переходов

Один из вариантов описания цепи Маркова, как и других графов - **матрица переходов** - таблица, где строка = текущее состояние, столбец = следующее, значение = вероятность перехода:

Из \ В	Patrol	Chase	Attack	Flee
Patrol	0.5	0.3	0.0	0.2
Chase	0.1	0.3	0.5	0.1
Attack	0.1	0.3	0.4	0.2
Flee	0.5	0.1	0.0	0.4

Два правила:

1. Сумма каждой строки = 1.0 (откуда-то мы точно перейдём).
2. Все значения ≥ 0 (отрицательных вероятностей не бывает).

Обратите внимание: из Patrol в Attack вероятность 0 - враг не может атаковать, не преследуя. Из Flee в Attack тоже 0 - убегающий не атакует. Это уже **дизайн поведения** через числа.

Базовая реализация

Данные хранятся как словарь: состояние -> список `(следующее_состояние, вероятность)`. Переход - это [взвешенный выбор](#) из списка текущего состояния:

Python C#

```python
transitions = {
    "patrol": [("patrol", 0.5), ("chase", 0.3), ("flee", 0.2)],
    "chase":  [("patrol", 0.1), ("chase", 0.3), ("attack", 0.5), ("flee", 
    "attack": [("patrol", 0.1), ("chase", 0.3), ("attack", 0.4), ("flee", 
    "flee":   [("patrol", 0.5), ("chase", 0.1), ("flee", 0.4)],
}

def next_state(current):
    options = transitions[current]
    r = random.random()
    cumulative = 0
    for state, prob in options:
        cumulative += prob
        if r < cumulative:
            return state
    # ... fallback to last option
```

По сути, это тот же подход, что и в лут-таблицах. Разница в том, что здесь выбор зависит от текущего состояния. Обернуть это в класс `MarkovChain` с методом `NextState(current)` - дело пяти минут.

Генерация имён

Ок. С поведением NPC разобрались - цепь заменяет if-else на таблицу вероятностей. Но цепи Маркова отлично генерируют и текст. Идея: считаем, какие буквы встречаются после каких, и генерируем новые слова по тем же закономерностям.

Допустим, у нас есть список фэнтезийных имён: Aragorn, Gandalf, Legolas, Arwen, Galadriel. Строим таблицу биграмм: после 'A' чаще всего идёт 'r', после 'r' - 'a' или 'o', и т.д. Затем генерируем новое имя, выбирая каждую следующую букву по этим вероятностям.

Алгоритм в два этапа:

Обучение. Берём список имён, оборачиваем каждое в маркеры начала/конца: `^aragorn$`. Проходим по всем парам соседних букв и считаем, сколько раз после `a` встречалось `r`, после `r` - `a` или `o`, и т.д. Потом нормализуем счётчики в вероятности - получаем ту же матрицу переходов, только состояния = буквы.

Генерация. Стартуем с `^`. Выбираем следующую букву взвешенным выбором из таблицы для текущей. Повторяем, пока не выпадет `$` (конец имени).

```
training: ["Aragorn", "Gandalf", "Legolas", ...]

^ -> a:0.4, g:0.3, l:0.1, b:0.1, e:0.05, f:0.05
a -> r:0.4, l:0.2, n:0.15, g:0.1, ...
r -> a:0.3, o:0.3, i:0.2, $:0.2
...

generate: ^ -> a -> r -> o -> n -> d -> $  =>  "Arond"
```

Маркер `$` важен - без него генератор не знает, когда остановиться, и выдаёт бесконечные строки. Маркер `^` нужен, чтобы имена начинались с правдоподобных букв, а не с середины слова.

Результат: имена, которые *звучат* как фэнтезийные, но не копируют исходные. Тот же подход работает для названий городов, заклинаний, предметов - любого текста, где важен "стиль", а не точность.

Цепи высших порядков

Биграммы (порядок 1) смотрят на одну предыдущую букву. Результат часто бессвязный: "Xgf", "Aaaa". Триграммы (порядок 2) смотрят на две предыдущие буквы - и результат сразу становится осмысленнее.

```
Порядок 1: следующая буква зависит от 1 предыдущей
Порядок 2: следующая буква зависит от 2 предыдущих
Порядок 3: следующая буква зависит от 3 предыдущих
```

Грубо говоря, чем выше порядок - тем "умнее" генерация, но тем больше данных нужно для обучения. При порядке 5 на корпусе из 10 имён модель будет просто копировать исходные имена целиком - не хватает комбинаций для разнообразия.

Компромисс "память vs разнообразие" - фундаментальный. Слишком мало памяти = бессмыслица. Слишком много = копирование. Оптимальный порядок подбирается экспериментально - для имён это обычно 2-3.

Поглощающие состояния

Некоторые состояния - "ловушки": попав в них, система больше не выходит. Такое состояние называется **поглощающим** (absorbing).

В играх это:

- **Game Over** - из смерти нет обратного пути
- **Победа** - уровень пройден

- **Поломка оружия** - если не предусмотрен ремонт

Матрица с поглощающим состоянием:

Из \ В	Alive	Wounded	Dead
Alive	0.7	0.3	0.0
Wounded	0.2	0.5	0.3
Dead	0.0	0.0	1.0

`Dead -> Dead` с вероятностью 1.0 - это и есть поглощение. Т.е. вопрос не *умрёт ли* персонаж, а **сколько ходов в среднем до этого?** Для геймдизайна полезно: можно посчитать "сколько боёв в среднем выживет персонаж".

Можно посчитать это точно. Берём подматрицу `Q` - только переходы между "живыми" состояниями (без Dead):

```
Q = [[0.7, 0.3],    -- Alive -> Alive, Alive -> Wounded
     [0.2, 0.5]]    -- Wounded -> Alive, Wounded -> Wounded
```

Дальше - линейная алгебра. `I` - единичная матрица (1 на диагонали, 0 в остальных местах). Вычисляем `(I - Q)` и обращаем её - получаем **фундаментальную матрицу** `N = (I - Q)^(-1)`. Сумма по строке `N` - среднее число шагов до поглощения (смерти).

```
I - Q = [[0.3, -0.3],
         [-0.2, 0.5]]

N = (I - Q)^(-1) = [[5.56, 3.33],  -- starting from Alive:  ~5.6 visits to Ali
                    [2.22, 3.33]]  -- starting from Wounded: ~2.2 visits to A

row_sums(N) = [8.9, 5.6]
-- from Alive:   ~8.9 steps until death
-- from Wounded: ~5.6 steps until death
```

Обращение матрицы - тема из курса линейной алгебры. Для студенческого проекта достаточно `numpy.linalg.inv()` или оценки Монте-Карло (прогнать 10 000 симуляций и посчитать среднее).

Плюсы и минусы

Плюсы:

- **Простота.** Матрица переходов + взвешенный выбор = 20 строк кода.
- **Обучаемость.** Можно не придумывать вероятности вручную, а **посчитать** их из данных (логи игроков, текст, музыка).
- **Лёгкость.** В отличие от нейросетей, цепь Маркова не требует GPU, обучения на тысячах примеров и тонкой настройки.
- **Управляемость.** Каждая вероятность - ручка, которую дизайнер крутит напрямую. Хочешь более агрессивного врага? Увеличь `Chase -> Attack`.

Минусы:

- **Нет памяти (порядок 1).** Цепь "не помнит", что было два хода назад. Враг может бесконечно дёргаться между Patrol и Chase - у цепи нет понятия "ты уже пятый раз это делаешь".
- **Нет дальней структуры.** Сгенерированные имена звучат правдоподобно на 2-3 буквы, но длинные тексты - бессмыслица. Для связного текста нужны более мощные модели.
- **Нужны данные.** Для обучения нужен корпус - список имён, логи поведения, тексты. Без данных остаётся ручной подбор вероятностей.

Свойство Маркова (memoryless): будущее зависит только от настоящего, не от прошлого. Это и сила (простота), и слабость (нет контекста). Повышение порядка цепи - компромисс: больше памяти, но экспоненциально больше состояний.

За пределами NPC

Цепи Маркова - фундамент, на котором стоят серьёзные вещи:

- **Скрытые марковские модели (HMM)** - состояния не видны напрямую, мы наблюдаем только "выходы". Используются в распознавании речи: скрытые состояния = фонемы, наблюдения = звуковые фрагменты.
- **PageRank** - Google ранжирует страницы, моделируя случайного пользователя, который кликает по ссылкам. Интернет = граф состояний, клики = переходы, PageRank = стационарное распределение цепи Маркова.
- **MCMC (Markov Chain Monte Carlo)** - метод сэмплирования из сложных распределений. Фундамент байесовской статистики и физических симуляций.
- **Reinforcement Learning** - MDP (Markov Decision Process) = цепь Маркова + действия агента + награды. Тот же формализм, но агент **выбирает** переходы, а не подчиняется фиксированным вероятностям.

Если ваш ИИ врага на конечном автомате ведёт себя слишком предсказуемо - замените детерминированные переходы на вероятностные. Это буквально одна строка: вместо `if see_player: state = CHASE` пишем взвешенный выбор. Результат: враг становится менее "читаемым" и более живым.

Что дальше?

Что	Где подробнее
Детерминированные автоматы (FSM)	Конечные автоматы
ИИ врагов: Patrol / Chase / Attack	Простой ИИ врагов
Seed, PRD, shuffle bag - другие подходы к случайности	Случайность и вероятность
Деревья решений вместо вероятностей	Деревья поведений
Минимакс и MCTS - поиск по дереву игр	Минимакс
Генерация уровней алгоритмически	Процедурная генерация
Направленное случайное блуждание - простейшая цепь	Случайное блуждание
Нейросети вместо ручных вероятностей	Нейросети для игр

Powered by Wiki.js